

Tabla de contenido

Variables de Entorno y Variables de la Shell.....	1
Comprimir y descomprimir.....	4
.tar (tar).....	4
.tar.gz – .tar.z – .tgz (tar con gzip).....	4
.gz (gzip).....	4
.bz2 (bzip2)	4
.tar.bz2 (tar con bzip2).....	4
.zip (zip).....	4
.lha (lha).....	4
.zoo (zoo)	5
.rar (rar)	5
Actividades.....	5

Variables de Entorno y Variables de la Shell

El shell dispone de un mecanismo para definir variables que se pueden utilizar para almacenar información usada por los programas del sistema o para uso propio. Algunas variables las utiliza el propio shell u otros programas de UNIX. Se pueden definir otras para ser usadas por el propio usuario.

Muchas utilidades UNIX, incluyendo el shell, necesitan información acerca del usuario y de qué pretende hacer para poder realizar bien el trabajo. Por ejemplo, para ejecutar un comando o programa en UNIX, es necesario especificar el nombre absoluto del programa o comando, o de lo contrario el sistema no sabrá qué comando ejecutar. En vez de forzar al usuario a proporcionar esa información cada vez que teclea un comando, UNIX utiliza las variables de entorno como un mecanismo con el que usuario pueda almacenar esa información y despreocuparse de ella. En el caso que nos ocupa, la variable de entorno PATH, lista todos aquellos directorios que almacenan comandos o programas que el usuario desea ejecutar. Cuando se invoca a un comando, el shell recorre cada directorio de la variable PATH para buscar ese comando. Probablemente, UNIX no necesitaría una variable de entorno si todos los comandos estuvieran en el mismo directorio.

La diferencia entre variables de entorno y variables de shell, es que las variables de shell son locales a una shell, mientras que una variable de entorno se hereda por cualquier otro programa que se está ejecutando. Las variables de shell son útiles únicamente en el shell actual, pero las variables de entorno son exportadas automáticamente quedando disponibles de forma global. Por ejemplo, las

variables de shell pueden estar disponibles únicamente a un script particular en el que están definidas, mientras que las variables de entorno se utilizan por cualquier script de shell, utilidad de correo, editor, etc. que lo invoque.

Las variables de entorno se asignan del siguiente modo:

```
$ setenv NOMBRE valor          # En el shell C
$ NOMBRE = valor; export NOMBRE # En los shell de Bourne o Korn
```

Por convenio, las variables de entorno se denotan en mayúsculas. El usuario puede crear sus propias variables de entorno o puede utilizar variables de entorno predefinidas. Algunas de estas variables predefinidas son las siguientes:

HOME	Especifica el directorio de arranque inicial el usuario, el directorio de trabajo.
SHELL	Almacena el nombre del programa de shell del usuario.
PATH	Lista, en orden y separados por dos puntos, los directorios en los que el shell busca para encontrar el programa a ejecutar cuando se teclea un comando.
TERM	Especifica el tipo de terminal de usuario. La utilizan <code>vi</code> y otros comandos orientados a pantalla.
PWD	Contiene el nombre de camino absoluto del directorio actual. La actualiza el comando <code>cd</code> .

Cuando un usuario entra en el sistema, UNIX fija automáticamente muchas de estas variables.

Las variables de shell vienen a ser una generalización de las variables de entorno. Las variables de shell pertenecen al shell y se comportan de forma similar a como lo hacen las variables en otros lenguajes. En términos de programación puede pensarse que las variables de entorno son variables globales mientras que las variables de shell son variables locales. Por convención las variables de shell se denotan en minúsculas. Para asignar valores a variables de shell puede utilizarse el comando:

```
$ set nombre=valor          # Shell C
$ nombre=valor              # Shell Bourne o shell Korn
```

Como caso especial, si se omite el valor, se asigna un valor `null` a la variable. Por ejemplo, los siguientes comandos son válidos:

```
$ set nombre                # Shell C (csh)
$ nombre=                   # Shell Bourne (bash) o Korn (ksh)
```

Obsérvese que no es lo mismo asignar un valor nulo a la variable que borrar la variable. Algunos programas miran si la variable existe o no sin preocuparse de

su contenido. Si se desea que la shell se olvide de que una variable existió, se utiliza el comando `unset`. Desafortunadamente, las shell de Bourne (bash) más antiguas no disponen de un comando de estas características.

```
$ unset nombre
```

El comando `set` mostrará tanto las variables locales como globales a diferencia de `env` y `printenv`.

Para modificar el valor de una variable de entorno durante la ejecución de una instancia del shell se utilizan los siguientes comandos. para que una variable forme parte del entorno debemos usar el comando `export`:

```
$ var=value  
$ export VAR=valor
```

En algunos sistemas no disponemos de `export`, utilizamos `setenv`

```
$ setenv VAR valor
```

Hay varios métodos para añadir variables de entorno en linux. Normalmente existen diferentes scripts que se ejecutan en diferentes eventos, por ejemplo, cuando un usuario se loguea en el sistema se ejecuta un archivo llamado `.bashrc`. Este es un archivo de sistema que se encuentra en la carpeta del usuario `/home/usuario/.bashrc`

Este script es un buen lugar para añadir la variable de entorno. Aunque también existen otros sitios.

En este ejemplo veremos cómo añadir una ruta a la variable de entorno `PATH`, que es una variable común para asignar rutas de aplicaciones y comandos que se instalan en el sistema.

Para evitar sobrescribir esta variable (es posible que se haya definido en otro sitio), podemos hacerlo de añadiendo esta línea al final del archivo

```
export PATH=$PATH:/directorio1:/directorio2:...:/directorioN
```

Para que los cambios surtan efecto podemos reiniciar sesión con ese usuario al que le hemos editado el archivo o bien podemos usar el comando de linux `source`

```
source /home/nombreusuario/.bashrc
```

Para probar que se ha guardado correctamente podemos escribir:

```
$PATH
```

Comprimir y descomprimir

Comandos de la consola para generar los archivos con extensiones:

.tar (tar)

Empaquetar	tar cvf archivo.tar /archivo/mayo/*
Desempaquetar	tar xvf archivo.tar
Ver el contenido (sin extraer)	tar tvf archivo.tar

.tar.gz – .tar.z – .tgz (tar con gzip)

Empaquetar y comprimir	tar czvf archivo.tar.gz /archivo/mayo/*
Desempaquetar y descomprimir	tar xzvf archivo.tar.gz
Ver el contenido (sin extraer)	tar tzvf archivo.tar.gz

.gz (gzip)

Comprimir	gzip -q archivo (El archivo lo comprime y lo renombra como “archivo.gz”)
Descomprimir	gzip -d archivo.gz (El archivo lo descomprime y lo deja como “archivo”)
Nota: gzip solo comprime archivos, no directorios	

.bz2 (bzip2)

Comprimir	bzip2 archivo bunzip2 archivo (El archivo lo comprime y lo renombra como “archivo.bz2”)
Descomprimir	bzip2 -d archivo.bz2 bunzip2 archivo.bz2 (El archivo lo descomprime y lo deja como “archivo”)
Nota: bzip2 solo comprime archivos, no directorios	

.tar.bz2 (tar con bzip2)

Comprimir	tar -c archivos bzip2 > archivo.tar.bz2
Descomprimir	bzip2 -dc archivo.tar.bz2 tar -xv tar jvxf archivo.tar.bz2 (versiones recientes de tar)
Ver contenido	bzip2 -dc archivo.tar.bz2 tar -tv

.zip (zip)

Comprimir	zip archivo.zip /mayo/archivos
Descomprimir	unzip archivo.zip
Ver contenido	unzip -v archivo.zip

.lha (lha)

Comprimir	lha archivo.lha /mayo/archivos
Descomprimir	lha -x archivo.lha
Ver contenido	lha -v archivo.lha lha -l archivo.lha

.zoo (zoo)

Comprimir	zoo -a archivo.zoo /mayo/archivos
Descomprimir	zoo -x archivo.zoo
Ver contenido	zoo -v archivo.zoo zoo -L archivo.zoo

.rar (rar)

Comprimir	rar -a archivo.rar /mayo/archivos
Descomprimir	rar -x archivo.rar
Ver contenido	rar -v archivo.rar rar -l archivo.rar

Actividades

1. Crea una variable local saludo que contenga el saludo “hola que haces”
2. Indica el comando que tengo que realizar para ver el contenido (echo)
3. Ejecuta el mismo comando en otra terminal ¿Qué información nos muestra?
4. Indica el comando para mostrar las variables locales (ejecutarlo en la terminal donde ejecutaste el ejercicio 1)
5. Indica el comando para ver las variables locales y de entorno (ejecutarlo en la terminal donde ejecutaste el ejercicio 1).
6. Indica el comando que podemos utilizar para mostrar el contenido, únicamente, de las variables PATH, HOME, SHELL.
7. Crea una variable SALUDO a partir de la variable creada en el ejercicio 1. Conviértela en una variable de entorno.
8. Indica el comando que muestra la información de las variables HOME y TEMP (o TMP, según tu distribución). ¿Qué información nos da cada una?
9. Quiero obtener una copia de seguridad de mi carpeta personal en el directorio temporal en un fichero con el siguiente nombre **seguridad.tar.gz**. Indique el comando.